

Magic Wand

1 Explanation

1.1 Overview

I basically just filled in all of the functions that were given to me in the python notebook. Nothing too out of the ordinary there.

We started off by finding the edges of a grayscale processed image with `preprocess_image()`. Instead of doing our own custom edge detection like in lab, we just used `skimage.feature.canny()`.

Once we did that, we also used another function `hough_circle_peaks()` to circle any “circles” found in the image. From there we used the center of that circle for the “little x” and “little y”; this was used to estimate the depth and world coordinates.

Drawing the ball and circle was very simple. So was the 2d line. Once we had the center we could also draw a cube around the points, calculating the theoretical world coordinates of the corners, and then projecting them to image coordinates, and then drawing lines between them. The separation of 2d and 3d functions made this easy to think about.

1.2 Where I Deviated

There were some instructions that I didn’t follow, and there were some functions that I changed.

The instructions said to use `plt.plot()` directly in the functions, however it made more sense to take in a given frame, and draw on that frame, and return a new frame. I thought this was cleaner, and it just made more sense to me. I think there are times and places for Python notebooks but I try and stay away from modifying global values. It just feels weird to me.

Once I have those functions that takes a frame in, draws on it, then returns it then I can call all of the draw functions in the final code block that uses the video player.

2 Issues

I still had issues with the circle detection; if the circle was not detected properly, then the rest of the functions would be completely off and irrelevant. You can see this in the drawn circle and cube. I will probably have to work with the `sigma`, `low_threshold` and `high_threshold` more to get more accurate readings.

3 Demo

The last code block has three demos that perform the requirements listed in the GitHub repo. All of the constants for the demo that is turned in is for the `red_and_blue.mov` video.

- Show the result of `preprocess_image`.
- Show the result of `detect_circles` and `calculate_ball_position` (use `draw_ball` to draw the ball on the image.) You should draw all balls detected, even if there is more than one.
- Show the result of `draw_bounding_cube`.
- Concatenate all 3D ball positions detected across all frames into an array and show the result as a 3D scatter plot.

4 Sources Used

I looked up how to add a text box to a random point in a `numpy` image on google and some article from GeeksForGeeks showed up and I copied some code from there.

I also copied the code for finding the `hough_circle_peaks()` in the demo which was linked in the GitHub instructions.

I also had a hard time finding good values for `skimage.features.canny()`. I asked Gemini what would be good values for `sigma`, `low_threshold` and `high_threshold`.

Other than that, I just used the pinhole equations from the week 3 slides.

5 Evaluation Questions

1. *Does the ball detector seem accurate? Explain when it fails to detect the ball and hypothesize why it fails.*

When I was playing around with the `sigma` `low_threshold` and `high_threshold` values, there were times when the thumbnail ended up being “detected” as a circle. This approximation is probably looking for circular objects on the 2d image, so I could see how a thumb could be seen as “relatively” circular.

2. *Would it be possible to correctly rotate the 3D box according to the ball’s orientation? Why or why not? If not, how could we physically modify the magic wand so that we could calculate the correct rotation of the box?*

If the ball was just on it’s own, then probably not; however, we do know that the handle of the wand is fixed to the ball, so we could do some sort of rectangle detection to get some vector from the handle. Then we would just put some transformation on all three of the points based on the angle that the wand is at in relation to “level”.

When the user rotates the handle while keeping the ball completely still, then we don't have any knowing of how much "z" rotation was applied. To get rotation in the "z axis", we could add another ball to the side of the handle, making somewhat of a 90 degree angle with the two balls. Then if rotation was applied in any direction, we could calculate the vectors and adjust the box accordingly. We might also want more balls just in case the handle covers one of them.